



QPOS Android SDK Integration Guide

Dspread Technology (Beijing) Inc

July 22, 2026

Contents

1 Document Overview	3
2 Preface	4
2.1 Purpose & Scope	4
2.2 Target Audience	4
3 Glossary & Definitions	5
4 SDK Overview	6
4.1 Implement SDK Method	6
4.1.1 Method 1: Import SDK AAR Package	6
4.1.2 Method 2: Import SDK from Maven Repository	7
4.2 Supported System Versions & IDE	7
4.3 Android Permission	7
4.4 Trade Flow Diagram	7
5 API Reference	9
5.1 Connection APIs	9
5.2 Transaction APIs	20
5.3 EMV Configuration APIs	37
5.4 Key Management APIs	38
5.5 Firmware Update APIs	39
5.6 Device Information APIs	41
A Enum Definitions	43
A.1 CardTradeMode	43
A.2 DoTradeResult	44
A.3 EmvOption	44
A.4 PinError	44
A.5 TransactionResult	45
A.6 UpdateInformationResult	45
A.7 Error	45

1 Document Overview

This document serves as the official integration guide for the QPOS Android SDK. It provides detailed, step-by-step instructions for developers to connect QPOS payment terminal devices (including QPOS, QPOS Reader, and SPOS) to Android applications.

The content covers core topics such as system requirements, SDK API usage, transaction processes, and common operation guidelines. The goal is to help developers quickly complete integration and ensure the payment function is secure and stable.

2 Preface

2.1 Purpose & Scope

Purpose: This document standardizes the integration process of QPOS Android SDK, clarifies the usage of SDK APIs, transaction logic, and troubleshooting methods, and ensures that developers can correctly connect QPOS devices to Android applications to achieve secure and stable payment transactions.

Scope: Covers QPOS series devices (QPOS, QPOS Reader, SPOS) and their Android SDK. It includes system environment requirements, SDK framework, permission configuration, API parameter descriptions, transaction flow diagrams, and delegate method callbacks, but does not involve hardware maintenance of QPOS devices.

2.2 Target Audience

- Android application developers responsible for integrating mobile payment functions.
- Technical support engineers who provide QPOS SDK integration guidance.
- Testers who verify the compatibility and stability of QPOS device connections.

3 Glossary & Definitions

Term	Definition
SDK	Software Development Kit (a toolset for integrating QPOS devices with applications)
QPOS	Mobile payment card reader with PIN pad (supports multiple communication modes)
QPOS Reader	Mobile payment card reader without PIN pad (supports Audio/USB communication)
SPOS	Mobile payment card reader with PIN pad and touch screen (supports signature capture)
EMV	Europay, MasterCard, Visa (a global standard for chip card payments)
ICC	Integrated Circuit Card (commonly known as "chip card")
APDU	Application Protocol Data Unit (data interaction protocol between card and terminal)
DUKPT	Derived Unique Key Per Transaction (a key management standard for secure transactions)
KSN	Key Serial Number (unique identifier for key derivation)
IPEK	Initial PIN Encryption Key (initial key for PIN encryption)
TLV	Tag-Length-Value (a data encoding format used in EMV transactions)

4 SDK Overview

This article is the integration development guide document for the Dspread unified SDK standard. It is intended to guide users on how to use the SDK, assuming that the reader is already familiar with the basic usage of the IDE (Android Studio) and has a certain foundation in Android programming.

4.1 Implement SDK Method

There are two methods to integrate the QPOS Android SDK:

4.1.1 Method 1: Import SDK AAR Package

1. **Download the SDK AAR package:** Obtain the SDK AAR package from the designated source ([SDK aar link](#))
2. **Place the AAR Package in the Project:** Copy the downloaded AAR package into the libs folder of your Android project. (e.g., dspread_pos_sdk_8.4.1.aar)

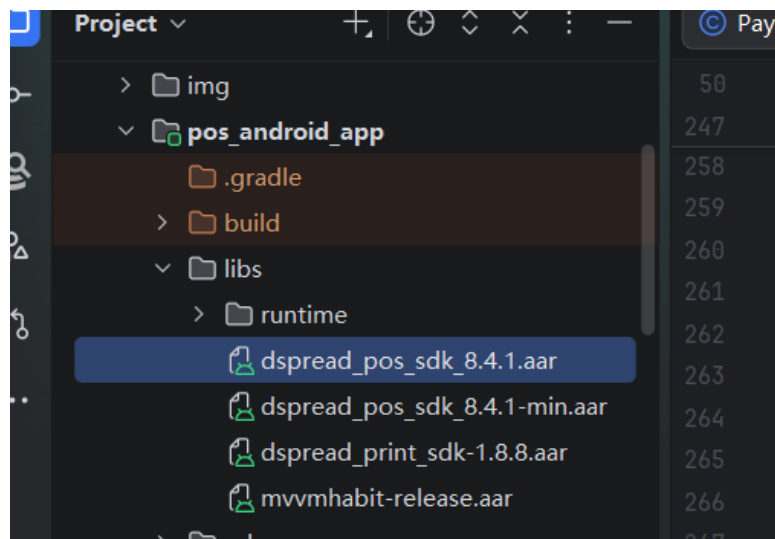


Figure 4.1.: AAR stroe

3. **Configure the App-Level Build File:** Add the following code to the app/build.gradle (or app/build.gradle.kts) file to import the AAR package:

```
implementation files('libs/dspread_pos_sdk_8.4.1.aar')
```

4.1.2 Method 2: Import SDK from Maven Repository

1. Check the latest SDK version

Please access the following page to view the latest SDK version:

<https://gitlab.com/dspread/android/-/packages>

2. Add Maven Repository

Add the following repository configuration to your project's Gradle file:

```
maven {  
    url 'https://gitlab.com/api/v4/projects/4128550/packages/maven'  
}
```

3. Add SDK Dependency

```
implementation 'com.dspread.library:dspread_pos_sdk:8.4.1'
```

4.2 Supported System Versions & IDE

- Android API 19(Android 4.4) and above
- Android Studio and IntelliJ supported

4.3 Android Permission

```
<uses-permission android:name="android.permission.BLUETOOTH" />  
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />  
  
<!-- Required permissions for Android 12 and above versions -->  
<uses-permission  
    android:name="android.permission.BLUETOOTH_SCAN"  
    android:usesPermissionFlags="neverForLocation"  
    tools:targetApi="s" />  
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />  
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />  
  
<!-- Location permissions -->  
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

4.4 Trade Flow Diagram

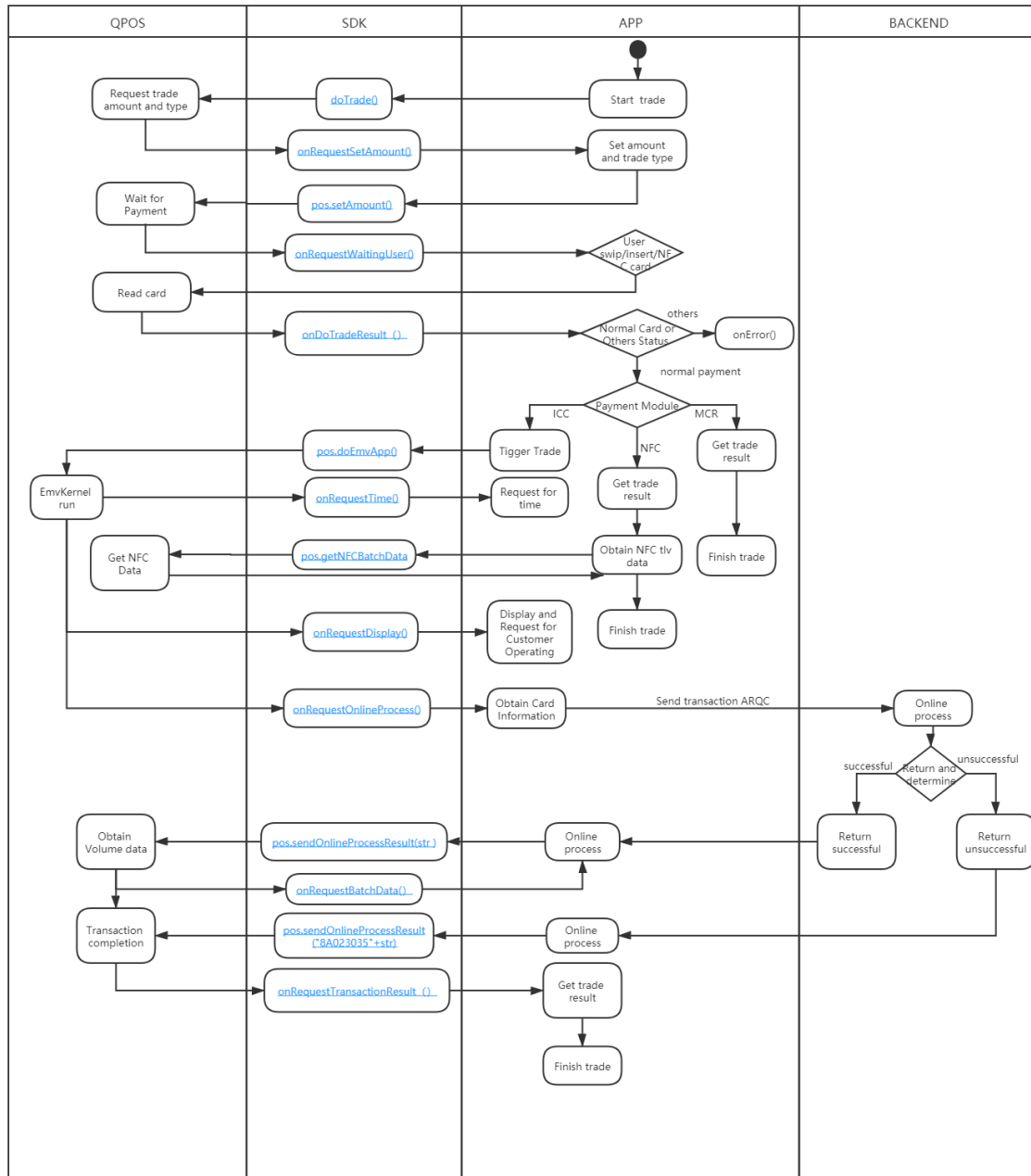


Figure 4.2.: Trade Flow Diagram

5 API Reference

5.1 Connection APIs

getInstance

Signature

```
QPOSService getInstance(Context ctx, CommunicationMode mode)
```

Description

Get the `QPOSService` instance according to communication mode.

It should be called first in the app to init the `QPOSService` class.

Parameters

Name	Description
ctx	Android context instance
mode	CommunicationMode.UART, BLUETOOTH, BLUETOOTH_BLE, USB_OTG_CDC_ACM

Outputs

Return the `QPOSService` instance

Example

```
QPOSService pos = QPOSService.getInstance(  
    MainActivity.this,  
    CommunicationMode.UART  
)
```

initListener

Signature

```
void initListener(QPOSServiceListener listener)
```

Description

Initializes the QPOSServiceListener callback: the app must register all the sub-functions in QPOSServiceListener.

Since the handler is not initialized in the SDK, the SDK invokes all callback methods on background threads by default. Therefore, the application must ensure that all UI operations inside the callbacks are performed on the main UI thread (e.g., by using runOnUiThread, Handler, or equivalent). The SDK does not handle thread switching automatically.

Parameters

Name	Description
listener	An instance of QPOSServiceListener

Outputs

None

initListener

Signature

```
void initListener(Handler handler, QPOSServiceListener listener)
```

Description

Initializes the QPOSServiceListener callback: the app must register all the sub-functions in QPOSServiceListener.

The handler is initialized within the SDK, callback methods are dispatched through the handler thread. Therefore, UI operations can be performed directly inside the callbacks without additional thread switching.

Parameters

Name	Description
handler	The handler to perform UI operation
listener	An instance of QPOSServiceListener

Outputs

None

openUart

Signature

```
void openUart()
```

Description

Open the SPOS devices.

After init the QPOSService, then could use this API to connect to SPOS devices.

The connect result will be triggered in callback onRequestQposConnected, onRequestNoQposDetected, which could be found at the end of this section.

Parameters

None

Outputs

None

closeUart

Signature

```
void closeUart()
```

Description

Close the SPOS devices.

It's used to disconnect. The disconnect result will be triggered in callback `onRequestQposDisonnected`, which could be found at the end of this section.

Parameters

None

Outputs

None

scanQPos2Mode

Signature

```
boolean scanQPos2Mode(Context context, long time)
```

Description

Scan bluetooth devices.

To use bluetooth, it needs app call this api to obtain the deviecs after get the permission of bluetooth. The scan results will be delivered through the registered callback `onDeviceFound`.

Parameters

Name	Description
context	context instance
time	the timeout for scan device, in seconds. The max time limit depends on bluetooth version supported on the phone.

Outputs

None

onDeviceFound

Signature

```
void onDeviceFound(BluetoothDevice device)
```

Description

Callback of bluetooth scan response.

When a bluetooth device is found, it will be triggered and return the device.

Parameters

Name	Description
device	Android standard class <code>BluetoothDevice</code> .

stopScanQPos2Mode

Signature

```
void stopScanQPos2Mode()
```

Description

Stop scan bluetooth devices.

This interface should be called before connecting the required device.

Parameters

None

Outputs

None

connectBluetoothDevice

Signature

```
boolean connectBluetoothDevice(boolean auto,int bondtime,String blueToothAddress)
```

Description

Connect normal bluetooth devices.

After init the `QPOSService`, then could use this API to connect to bluetooth devices.

The connect result will be triggered in callback `onRequestQposConnected`, `onRequestNoQposDetected`, which could be found at the end of this section.

Parameters

Name	Description
auto	if is true,can pair the device automatically,else is false,need pair the device manually
bondtime	the bluetooth bond time, in seconds.
blueToothAddress	device bluetooth MAC address

Example

```
pos.connectBluetoothDevice(  
    true,  
    20,  
    "00:11:22:33:44:55"  
);
```

disconnectBT

Signature

```
void disconnectBT()
```

Description

Disconnect normal bluetooth device.

The disconnect result will be triggered in callback `onRequestQposDisonnected` , which could be found at the end of this section.

Parameters

None

Outputs

None

startScanQposBLE

Signature

```
void startScanQposBLE(int timeout)
```

Description

Scan BLE devices.

To enable BLE (Bluetooth Low Energy) functionality, this API must be called after the Bluetooth permission has been granted.

The scan results will be delivered through the registered callback `onDeviceFound`.

Parameters

Name	Description
timeout	the timeout for scan device, in seconds.

Outputs

None

onDeviceFound

Signature

```
void onDeviceFound(BluetoothDevice device)
```

Description

Callback of bluetooth scan response.

When a bluetooth device is found, it will be triggered and return the device.

Parameters

Name	Description
device	Android standard native class <code>BluetoothDevice</code> .

stopScanQposBLE

Signature

```
void stopScanQposBLE()
```

Description

Stop scan BLE devices.

This interface should be called before connecting the required device.

Parameters

None

Outputs

None

connectBLE

Signature

```
boolean connectBLE(String blueToothAddress)
```

Description

Connect BLE QPOS device.

After init the `QPOSService`, then could use this API to connect to BLE devices.

The connect result will be triggered in callback `onRequestQposConnected`, `onRequestNoQposDetected`, which could be found at the end of this section.

Parameters

Name	Description
blueToothAddress	device BLE MAC address

Outputs

None

Example

```
pos.connectBLE("00:11:22:33:44:55");
```

disconnectBLE

Signature

```
void void disconnectBLE()
```

Description

Disconnect BLE device.

The disconnect result will be triggered in callback `onRequestQposDisconnected`, which could be found at the end of this section.

Parameters

None

Outputs

None

openUsb

Signature

```
void openUsb(UsbDevice usbDevice)
```

Description

Open USB.

After init the `QPOSService`, then could use this API to connect to devices via USB.

The app could use Android native method to detect USB terminal, then use this API to connect POS.

The connect result will be triggered in callback `onRequestQposConnected`, `onRequestNoQposDetected`, which could be found at the end of this section.

Parameters

Name	Description
usbDevice	Android standard native class <code>UsbDevice</code>

Outputs

None

closeUsb

Signature

```
void closeUsb()
```

Description

Disconnect USB.

The disconnect result will be triggered in callback `onRequestQposDisconnected`, which could be found at the end of this section.

Parameters

None

Outputs

None

onRequestQposConnected

Signature

```
void onRequestQposConnected()
```

Description

Triggered when device connection succeeds.

onRequestQposDisconnected

Signature

```
void onRequestQposDisconnected()
```

Description

Triggered when device disconnects.

onRequestNoQposDetected

Signature

```
void onRequestNoQposDetected()
```

Description

Triggered when no QPOS device is detected.

5.2 Transaction APIs

setCardTradeMode

Signature

```
void setCardTradeMode(CardTradeMode cardTradeMode)
```

Description

Set the card reading mode allowed by the device.

This should be called before `doTrade()` .

Parameters

Name	Description
cardTradeMode	QPOSService.CardTradeMode.*

CardTradeMode.ONLY_INSERT_CARD,
ONLY_SWIPE_CARD,TAP_INSERT_CARD,
TAP_INSERT_CARD_NOTUP,
SWIPE_TAP_INSERT_CARD,
UNALLOWED_LOW_TRADE,SWIPE_INSERT_CARD,
SWIPE_TAP_INSERT_CARD_UNALLOWED_LOW_TRADE,
SWIPE_TAP_INSERT_CARD_NOTUP_UNALLOWED_LOW_TRADE,
ONLY_TAP_CARD,ONLY_TAP_CARD_QF,
SWIPE_TAP_INSERT_CARD_NOTUP,
SWIPE_TAP_INSERT_CARD_DOWN,
SWIPE_INSERT_CARD_UNALLOWED_LOW_TRADE,
SWIPE_TAP_INSERT_CARD_UNALLOWED_LOW_TRADE_NEW,
ONLY_INSERT_CARD_NOPIN,
SWIPE_TAP_INSERT_CARD_NOTUP_DELAY

Outputs

None

Example

```
pos.setCardTradeMode(CardTradeMode.SWIPE_TAP_INSERT_CARD);
```

doTrade

Signature

```
void doTrade(int timeout)
```

Description

Start card transaction.

The trade result will be delivered through the registered callback `onDoTradeResult`.

Parameters

Name	Description
timeout	timeout of trade, in seconds. Valid range: 1–255 seconds.

Outputs

None

Example

```
pos.doTrade(60);
```

onDoTradeResult

Signature

```
void onDoTradeResult(DoTradeResult result,Hashtable<String, String> decodeData)
```

Description

Callback of trade response.

Parameters

Name	Description
result	Card trade result:QPOSService.DoTradeResult.*
decodeData	the transaction data

DoTradeResult.MCR,ICC,NFC_ONLINE,NFC_OFFLINE,
NFC_DECLINED,PLS_SEE_PHONE

setAmount

Signature

```
void setAmount(String amount,String cashbackAmount,String  
currencyCode,TransactionType transactionType)
```

Description

Set the amount and transaction type required for EMV transaction.

App could call this before `doTrade()` or in the callback `onRequestSetAmount`.

Parameters

Name	Description
amount	Transaction amount in cents
cashbackAmount	Cashback amount in cents
currencyCode	ISO currency code
transactionType	QPOSService.TransactionType.*

TransactionType.GOODS, SERVICES, CASH, CASHBACK,
PURCHASE_REFUND, INQUIRY, TRANSFER, ADMIN,
CASHDEPOSIT, PAYMENT, PBOCLOG, SALE,
PREAUTH, ECQ_DESIGNATED_LOAD,
ECQ_UNDESIGNATED_LOAD, ECQ_CASH_LOAD,
ECQ_CASH_LOAD_VOID, ECQ_INQUIRE_LOG,
REFUND, UPDATE_PIN, SALES_NEW,
NON_LEGACY_MONEY_ADD, LEGACY_MONEY_ADD,
BALANCE_UPDATE, MINI_STATEMENT,
CUSTOMIZE, BALANCE

Outputs

None

Example

```
pos.setAmount(  
    "1000",  
    "",  
    "156",  
    TransactionType.GOODS  
);
```

onRequestSetAmount

Signature

```
void onRequestSetAmount()
```

Description

Callback requesting application to set amount.

This will be triggered automatically after `doTrade()` .

cancelSetAmount

Signature

```
void cancelSetAmount()
```

Description

Cancel the process about setting amount.

This will also cancel the current transaction.

Parameters

None

Example

```
pos.cancelSetAmount();
```

onRequestSetAmount

Signature

```
void onRequestSetAmount()
```

Description

Callback requesting application to set amount.

This will be triggered automatically after `doTrade()` .

sendTime

Signature

```
void sendTime(String terminalTime)
```

Description

Send current time. The transaction needs this data to complete.

App could call this before `doTrade()` or in the callback `onRequestTime`.

Parameters

Name	Description
terminalTime	current trade time (format: yyyyMMddHHmmss)

Outputs

None

Example

```
pos.sendTime("20210228192610");
```

onRequestTime

Signature

```
void onRequestTime()
```

Description

Callback method of request user to set time.

This will be triggered automatically after `doTrade()`.

doEmvApp

Signature

```
void doEmvApp(EmvOption emvOption)
```

Description

Send the command as executing the EMV transaction flow to POS.

This should be called in callback `onDoTradeResult` when doing an ICC trade

Parameters

Name	Description
emvOption	QPOSService.emvOption.*

EmvOption.START,START_WITH_FORCE_ONLINE,START_WITH_FORCE_PIN,
START_WITH_FORCE_ONLINE_FORCE_PIN,
START_WITH_RETURN_ICC_CARD_NUMBER,
START_WITH_FORCE_ONLINE_RETURN_ICC_CARD_NUMBER,
START_WITH_FORCE_PIN_RETURN_ICC_CARD_NUMBER,
START_WITH_FORCE_ONLINE_FORCE_PIN_RETURN_ICC_CARD_NUMBER

Outputs

None

Example

```
pos.doEmvApp(EmvOption.START);
```

selectEmvApp

Signature

```
void selectEmvApp(int index)
```

Description

Select one application of the application list returned from EMV kernel, then set the application ID to EMV kernel.

This should be called when callback `onRequestSelectEmvApp` triggered.

Parameters

Name	Description
index	index of the list of application(starting from 0)

Outputs

None

Example

```
pos.selectEmvApp(1);
```

onRequestSelectEmvApp

Signature

```
void onRequestSelectEmvApp(ArrayList<String> appList)
```

Description

Callback method of request user to select emv app.

This callback will be triggered when a transaction requires multiple application selections.

Parameters

Name	Description
appList	the emv applications list.

cancelSelectEmvApp

Signature

```
void cancelSelectEmvApp()
```

Description

Cancel the process about setting the application.

This could be called in callback `onRequestSelectEmvApp`.

Parameters

None

Outputs

None

Example

```
pos.cancelSelectEmvApp();
```

onRequestSelectEmvApp

Signature

```
void onRequestSelectEmvApp(ArrayList<String> appList)
```

Description

Callback method of request user to select emv app.

This callback will be triggered when a transaction requires multiple application selections.

Parameters

Name	Description
appList	the emv applications list

sendPin

Signature

```
void sendPin(byte[] pin)
```

Description

Input the pin to CR100. The ISO format-4 PINblock will build in SDK internally.

This should be called when callback `onRequestSetPin` is triggered.

Parameters

Name	Description
pin	pin code length just allow 4-12

Outputs

None

Example

```
pos.sendPin("123456".getBytes());
```

onRequestSetPin

Signature

```
void onRequestSetPin()
```

Description

Callback of `sendPin()` / `sendCvmPin()`, this is used for CR100.

sendCvmPin

Signature

```
void sendCvmPin(String pin, boolean isEncrypted)
```

Description

Input the pin to CR100. App should build the PINblock first then use this API send to device.

This should be called when callback `onRequestSetPin` is triggered.

Parameters

Name	Description
pin	this pinblock should be the ISO format-4 pinblock which meets the MPoC protocol
isEncrypted	whether the pin is encrypted

Outputs

None

Example

```
String pinBlock = buildCvmPinBlock(  
    pos.getEncryptData(),  
    newPin  
); // build the ISO format4 pin block  
pos.sendCvmPin(  
    pinBlock,  
    true  
);
```

onRequestSetPin

Signature

```
void onRequestSetPin()
```

Description

Callback of `sendPin()` / `sendCvmPin()`, this is used for CR100.

cancelPin

Signature

```
void cancelPin()
```

Description

Set cancel input pin to CR100.

This could be called when callback `onRequestSetPin` triggered.

Parameters

None

Outputs

None

Example

```
pos.cancelPin();
```

onRequestSetPin

Signature

```
void onRequestSetPin()
```

Description

Callback of `sendPin()` / `sendCvmPin()` ,this is used for CR100.

pinMapSync

Signature

```
void pinMapSync(String datas, int timeout)
```

Description

Send the pin keyboard datas to Smart Device for pin input.

App should draw a keyboard according to dataList in the `onQposRequestPinResult` callback first, then calculate the position of each key and send to POS.

Parameters

Name	Description
datas	the string of keyboard pin coordinate position
timeout	timout of pin input, in seconds. Valid range:1–255 seconds.

Outputs

None

Example

```
pos.pinMapSync(
    value,
    30
);
```

onQposRequestPinResult

Signature

```
void onQposRequestPinResult(List<String> dataList,int offlineTime)
```

Description

This is used for smart POS to let app draw the keyboard and send key position to POS.

The callback of `pinMapSync()` .

Parameters

Name	Description
dataList	the random number value list returned by POS
offlineTime	remaining input times for offline pin

onReturnGetPinInputResult

Signature

```
void onReturnGetPinInputResult(int num,PinError error,int minLen,int maxLen)
```

Description

This is used for smart POS to return the number of pin inputs.

The app can display the number of PINs that have been entered based on the returned number.

Parameters

Name	Description
num	the num is the counter of your pin input
error	the PIN input result or error status returned by the device:QPOSService.PinError.*
minLen	the minimum allowed PIN length supported by the device
maxLen	the maximum allowed PIN length supported by the device

PinError.OK, TOO_LONG_MAX,
TOO_SHORT_ZERO, TOO_SHORT_MIN

sendOnlineProcessResult

Signature

```
void sendOnlineProcessResult(String tlv)
```

Description

Send the received online processing result to POS.

The result is obtained from issuer server. And it should contain tag 8A (Authorisation Response Code) and tag 91 (Issuer Authentication Data) and tag 71/72 (Issuer Script).

8A is necessary.

This should be called in callback `onRequestOnlineProcess`.

Parameters

Name	Description
tlv	online processing result

Outputs

None

Example

```
pos.sendOnlineProcessResult("8A023030");
```

onRequestOnlineProcess

Signature

```
void onRequestOnlineProcess(String tlv)
```

Description

EMV kernel request the online handler, should send the results back to EMV kernel by `sendOnlineProcessResult()`.

Parameters

Name	Description
tlv	contains the tag-length-value data structure returned by the EMV kernel

onRequestTransactionResult

Signature

```
void onRequestTransactionResult(TransactionResult transactionResult)
```

Description

Transaction result.

Parameters

Name	Description
transactionResult	QPOSService.TransactionResult.*

TransactionResult.APPROVED, TERMINATED,
DECLINED, CANCEL

onRequestBatchData

Signature

```
void onRequestBatchData(String tlv)
```

Description

Response of ICC transactions: when the transaction is finished, the batch data will be returned to the application.

Parameters

Name	Description
tlv	contains the tag-length-value data structure returned by the EMV kernel

onReturnReversalData

Signature

```
void onReturnReversalData(String tlv)
```

Description

When transaction is reversal, it will trigger and return the tlv data.

Parameters

Name	Description
tlv	contains the tag-length-value data structure returned by the EMV kernel

onEmvICCEExceptionData

Signature

```
void onEmvICCEExceptionData(String tlv)
```

Description

When transaction doesn't approve, it will trigger and return the tlv data.

Parameters

Name	Description
tlv	contains the tag-length-value data structure returned by the EMV kernel

getNFCBatchData

Signature

```
Hashtable<String, String> getNFCBatchData()
```

Description

Obtain the tlv data of NFC.

App could call this api after `onDoTradeResult`.

Parameters

None

Outputs

Return the TLV data of NFC.

Example

```
Hashtable<String, String> data = pos.getNFCBatchData();
```

5.3 EMV Configuration APIs

updateEMVConfigByXml

Signature

```
void updateEMVConfigByXml(String xmlContent)
```

Description

Update EMV configuration using XML file.

The update result will be delivered through the registered callback `onReturnCustomConfigResult`.

Parameters

Name	Description
xmlContent	parse EMV configuration file in xml format to get string

Outputs

None

Example

```
pos.updateEMVConfigByXml(new  
    ↳ String(FileUtils.readAssetsLine("emv_profile_tlv.xml")));
```

onReturnCustomConfigResult

Signature

```
void onReturnCustomConfigResult(boolean isSuccess,String result)
```

Description

Callback of updateEMVConfigByXml().

It will return the update result.

Parameters

Name	Description
isSuccess	EMV update result
result	Failure reason if update failed

5.4 Key Management APIs

updateKeyByTR_31

Signature

```
void updateKeyByTR_31(int keyIndex,String keyBlock)
```

Description

Update device key using TR31 key block.

The update result will be delivered through the registered callback `onReturnUpdateKeyByTR_31Result`.

Parameters

Name	Description
keyIndex	new key index, from 0 to 9
keyBlock	TR31 key block string

Outputs

None

Example

```
pos.updateKeyByTR_31(  
0,  
"B0080B1TX00E00007ADC3F4EC9DEC96726CAB25032AC86B71581429795F43E2B  
6B5DBCD646FE8F3A"  
);
```

onReturnUpdateKeyByTR_31Result

Signature

```
void onReturnUpdateKeyByTR_31Result(boolean result)
```

Description

Callback of `updateKeyByTR_31()`.

It will return the update result.

Parameters

Name	Description
result	True means success, false means failure

5.5 Firmware Update APIs

updatePosFirmware

Signature

```
int updatePosFirmware(byte[] data,String deviceAddress)
```

Description

Update the device firmware.

App needs to read and convert the .asc file into byte Array datas.

The update result will be delivered through the registered callback `onUpdatePosFirmwareResult`.

Parameters

Name	Description
data	the firmware asc file datas
deviceAddress	device address,like bluetooth connection,need input the device bluetooth mac address,other connection method can input"".

Outputs

Return int value :-1 means error,0 means can update.

Example

```
byte[] data = FileUtils.readAssetsLine(
    "CR100_master.asc",
    MainActivity.this
);
int updateResult = pos.updatePosFirmware(
    data,
    blueTootchAddress
);
```

onUpdatePosFirmwareResult

Signature

```
void onUpdatePosFirmwareResul(UpdateInformationResult result)
```

Description

Callback of `updatePosFirmware()` .
It will return the result of update.

Parameters

Name	Description
result	QPOSService.UpdateInformationResult.*

UpdateInformationResult.UPDATE_SUCCESS,UPDATE_FAIL,
UPDATE_PACKET_VEFIRY_ERROR, UPDATE_LOWPOWER,
USB_RECONNECTING

getUpdateProgress

Signature

```
int getUpdateProgress()
```

Description

Get the device firmware update progress.

It should be called in the child thread so that avoid to block the main thread.

Parameters

None

Outputs

Return the firmware update progress integer value.

5.6 Device Information APIs

getQposId

Signature

```
void getQposId()
```

Description

Get the device ID.

The result will be delivered through the registered callback `onQposIdResult`.

Parameters

None

Outputs

None

onQposIdResult

Signature

```
void onQposIdResult(Hashtable<String,String> posIdTable)
```

Description

Callback of `getQposId()`.

It will return the result in Hashtable format.

Parameters

Name	Description
posIdTable	device sn,psamId,merchantId,vendorCode

getQposInfo

Signature

```
void getQposInfo()
```

Description

Get the device info.

The result will be delivered through the registered callback `onQposInfoResult`.

Parameters

None

Outputs

None

onQposInfoResult

Signature

```
void onQposInfoResult(Hashtable<String,String> posInfoTable)
```

Description

Callback of `getQposInfo()`.

It will return the result in Hashtable format.

Parameters

Name	Description
posInfoTable	isSupportedTrack1, isSupportedTrack2, isSupportedTrack3, bootloaderVersion, firmwareVersion, isUsbConnected, batteryLevel, Voltage, hardwareVersion, SUB, updateWorkKeyFlag, isKeyboard, batteryPercentage, PCI_firmwareVersion, PCI_hardwareVersion, compileTime, Manufacturere, ModelInfor, OTAversion

A Enum Definitions

A.1 CardTradeMode

Enum	Description
ONLY_INSERT_CARD	Allow to insert card only.
ONLY_SWIPE_CARD	Allow to swipe card only.
TAP_INSERT_CARD	Allow to insert and tap card, need to click "UP" key to use NFC(Suitable for devices with physical keyboard).
TAP_INSERT_CARD_NOTUP	Allow to insert and tap card, no need to click "UP" key to use NFC.
SWIPE_TAP_INSERT_CARD	Allow to swipe, tap and insert card, need to click "UP" key to use NFC(Suitable for MPOS).
UNALLOWED_LOW_TRADE	Allow to insert,tap and swipe card, no need to click "UP" key to use NFC, and do not allow downgraded transactions.
SWIPE_INSERT_CARD	Allow to swipe and insert card.
SWIPE_TAP_INSERT_CARD_UNALLOWED_LOW_TRADE	Allow to swipe, tap and insert card, need to click "UP" key to use NFC(Suitable for MPOS), and do not allow downgraded transactions.
SWIPE_TAP_INSERT_CARD_NOTUP_UNALLOWED_LOW_TRADE	Allow to swipe, tap and insert card, no need to click "UP" key to use NFC, and do not allow downgraded transactions.
ONLY_TAP_CARD	Allow to tap card only, need to click "UP" key to use NFC(Suitable for MPOS).
ONLY_TAP_CARD_QF	Allow to tap card only(This is a customed Enum), need to click "UP" key to use NFC(Suitable for MPOS).
SWIPE_TAP_INSERT_CARD_NOTUP	Allow to swipe, tap and insert card,no need to click "UP" key to use NFC.
SWIPE_TAP_INSERT_CARD_DOWN	Allow to swipe, tap and insert card,need to click "UP" key to use NFC(Suitable for MPOS), and do not allow downgraded transactions.
SWIPE_INSERT_CARD_UNALLOWED_LOW_TRADE	Allow to swipe and insert card, do not allow downgraded transactions.
SWIPE_TAP_INSERT_CARD_UNALLOWED_LOW_TRADE_NEW	Allow to insert,tap and swipe card, no need to click "UP" key to use NFC, and do not allow downgraded transactions.

ONLY_INSERT_CARD_NOPI N	Deprecated.
SWIPE_TAP_INSERT_CARD _NOTUP_DELAY	Allow to swipe, tap and insert card, no need to click "UP" key to use NFC, NFC delay.

A.2 DoTradeResult

Enum	Description
MCR	Swipe card detected.
ICC	Contact chip card detected.
NFC_ONLINE	NFC card detected, go online.
NFC_OFFLINE	NFC card detected, offline approved.
NFC_DECLINED	NFC card transaction rejected.
PLS_SEE_PHONE	Please operate and confirm on customer mobile phone.

A.3 EmvOption

Enum	Description
START	Emv ICC start trade.
START_WITH_FORCE_ONLI NE	Start EMV transaction and force online authorization.
START_WITH_FORCE_PIN	Start EMV transaction and force cardholder PIN input.
START_WITH_FORCE_ONLI NE_FORCE_PIN	Start EMV transaction, force online auth and PIN entry.
START_WITH_RETURN_ICC _CARD_NUMBER	Emv ICC start trade and return full ICC card number after reading chip.
START_WITH_FORCE_ONLI NE_RETURN_ICC_CARD_N UMBER	Force online authorization and return ICC card number.
START_WITH_FORCE_PIN_ RETURN_ICC_CARD_NUMB ER	Force PIN input and return ICC card number after chip read.
START_WITH_FORCE_ONLI NE_FORCE_PIN_RETURN_I CC_CARD_NUMBER	Force online auth, force PIN entry, and return ICC card number.

A.4 PinError

Enum	Description
------	-------------

OK	Operation successful.
TOO_LONG_MAX	PIN length exceeds maximum limit.
TOO_SHORT_ZERO	PIN length is too short (restricted by zero).
TOO_SHORT_MIN	PIN length is too short (below minimum limit).

A.5 TransactionResult

Enum	Description
APPROVED	Trade approved.
TERMINATED	ICC trade terminated.
DECLINED	Trade declined.
CANCEL	Trade canceled.

A.6 UpdateInformationResult

Enum	Description
UPDATE_SUCCESS	Firmware update completed successfully.
UPDATE_FAIL	Firmware update process failed.
UPDATE_PACKET_VEFIRY_ERROR	Firmware file data packet verification error, it needs the supporter to provide new file.
UPDATE_LOWPOWER	Update interrupted due to insufficient battery power.
USB_RECONNECTING	USB communication disconnected, need to reconnect by manual.

A.7 Error

Enum	Description
NO_RESPONSE	Device not response.
MAC_ERROR	MAC error.
TIMEOUT	Command timeout.
CMD_NOT_AVAILABLE	Command not available.
DEVICE_RESET	Reset device.
UNKNOWN	Unknown error, need to check SDK log.
DEVICE_BUSY	Device busy.
DEVICE_IS_OCCUPIED	SPOS device uart is occupied by other task.
INPUT_OUT_OF_RANGE	Input out of range.
INPUT_INVALID_FORMAT	input invalid format.

INPUT_INVALID	Input invalid.
APDU_ERROR	APDU error.
APP_SELECT_TIMEOUT	Multiple application selection timeout.
ICC_ONLINE_TIMEOUT	ICC online timeout
ICC_EXISTS_ERROR	ICC card state abnormal
DEVICE_IN_BOOT_STATE	Device is in boot state, need to upgrade firmware.
REMOTE_SERVICE_EXCEPTION	Remote service call exception.
RECEIVE_PLAINT_IN_ENCRYPTION_MODE	Received a plain text command in encrypted mode(only used for SCRP).
RECEIVE_CIPHER_IN_NON_ENCRYPTION_MODE	Encrypted command received in non-encrypted mode(only used for SCRP).
DECRYPT_FAIL_IN_ENCRYPTION_MODE	Command decryption failed in encryption mode(only used for SCRP).
AID_MISSING	Required AID configuration missing.
API_NOT_AVAILABLE	Target API is unavailable
CAPK_FAIL	CAPK fail, CAPK not supported.
DEVICE_ERROR	Device error.
FALLBACK	ICC trade fall back to swipe transaction: might due to chip damage, abnormal card downgrade processing, or Device EMV config doesn't have card AID.
NFC_TERMINATED	NFC card reading failed.
CARD_REMOVED	Card removed during processing.
TRADE_LOG_FULL	Trade log storage full.
TRANSACTION_NOT_ALLOWED_AMOUNT_EXCEED	Transaction not allowed, amount limit exceeded
TRANS_TOKEN_INVALID	Transaction token invalid(used for SCRP).
CONTACTLESS_TRANSACTION_NOT_ALLOW	Contactless transaction not allow
CARD_BLOCKED_APDU_ERROR_6A81	Card blocked, APDU response code 6A81.
APP_BLOCKED_APDU_ERROR_6A83	APP BLOCKED, APDU response code 6A83.
TRY_ANOTHER_INTERFACE	Try another interface. Usually occurs when there is a problem with the contactless chip, requires ICC transactions.
MULTIPLE_CARDS	Multiple cards detected at the same time.
NOT_ICC	ICC card power on failure (IC card detected, but IC card chip abnormal).
NO_UPDATE_WORK_KEY	No update work key
NONE	No card detected

BAD_SWIPE	Abnormal swipe operation/ magnetic stripe abnormality.
EXPIRED_CERT	Certificate expired.
INVALID_TRUSTED_CERT	Trusted certificate invalid.
TRANSACTION_TIME_MIS SING_TIMEOUT	Transaction time not entried.